

An Animator-Friendly Chain Simulator for In-Game Animations at Blizzard Entertainment

FunShing Sin
Blizzard Entertainment
United States of America
fsin@blizzard.com

Vinod Melapudi
Blizzard Entertainment
United States of America
vmelapudi@blizzard.com

Parag Havaldar
Blizzard Entertainment
United States of America
phavaldar@blizzard.com

ABSTRACT

At Blizzard, the Animation department produces character animations which get evaluated in our video games by the game engines. These joint-based skeletal animations are mostly produced from motion capture systems or by artists using DCCs like Maya. However, to animate the accessories of a character like cloth and hair, artists prefer to employ simulation tools to achieve physically correct motions. Our artists explored a few existing solutions (e.g., Maya’s nCloth) to simulate these accessories, but were either limited by the difficulty in setup requirements, or were unsatisfied at the amount of time it took to get visibly acceptable simulation results. So, we developed a custom joint-chain solver inspired by various publications. In this talk, we will detail some of the existing solutions that were evaluated and present their limitations. Then we will illustrate our approach that helped overcome those limitations and extend the capabilities. And finally, we will discuss some use cases of our solution using our production characters and highlight how our artists benefited from employing the simulator.

CCS CONCEPTS

• Computer systems organization → Real-time systems.

KEYWORDS

Animations, Keyframes, Physics-based Simulations, Maya

ACM Reference Format:

FunShing Sin, Vinod Melapudi, and Parag Havaldar. 2023. An Animator-Friendly Chain Simulator for In-Game Animations at Blizzard Entertainment. In *Special Interest Group on Computer Graphics and Interactive Techniques Conference Talks (SIGGRAPH ’23 Talks)*, August 06–10, 2023, Los Angeles, CA, USA. ACM, New York, NY, USA, 2 pages. <https://doi.org/10.1145/3587421.3595447>

1 INTRODUCTION

Blizzard develops video games across multiple genres and our Animation department is tasked with producing various animations for the cinematics that get presented during the game’s runtime. Game engines predominantly support skeleton based character deformation. The characters of these cinematics are articulated using joint-based skeletal system driven by the control rigs which allows them to be animated. Artists choose to animate those characters

Permission to make digital or hard copies of part or all of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

SIGGRAPH ’23 Talks, August 06–10, 2023, Los Angeles, CA, USA

© 2023 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0143-6/23/08.

<https://doi.org/10.1145/3587421.3595447>

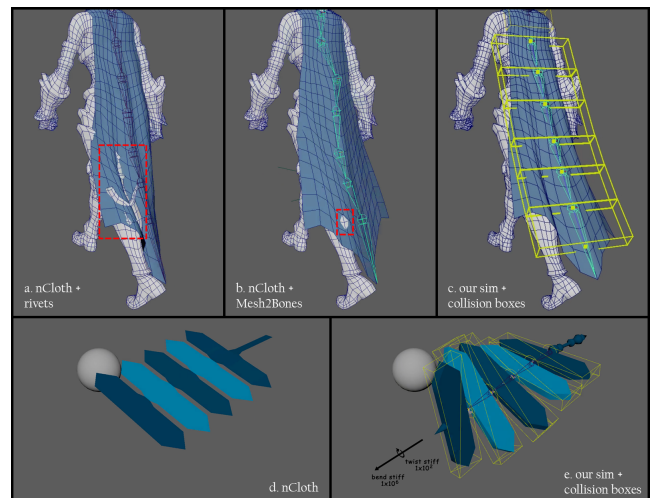


Figure 1: (a,b) Rivets and Mesh2Bones introduce penetrations (highlighted in red). (c) Our sim is collision-free. (d) Representing a chain using a one-piece cloth loses the ability to separate bending and twisting, (e) but our solver allows users to easily define bending/twisting differently.

by hand or employ motion capture system to retarget the motion. Many of our characters are highlighted by various features like clothes, chains, hair, etc. These features are also articulated by joints extended from the character’s primary skeleton. Most often, the motion of these features is not procured from the capture systems. And instead of hand animating the clothes and hair, animators prefer to use simulation solvers to produce physically realistic motions that can be replicated by the associated joint chains in the engine. Animators may choose to further edit the animation curves on the joints to achieve the desired artistic motion. Finally, the key frames on the joints of the character and its accessories are delivered to the game engine for skinning and rendering.

Our initial approach to simulate the cloth and hair meshes was to utilize Maya’s nCloth system. The pipeline involved setting up that mesh with various physical properties (mass, stretching, etc) and simulating using nCloth solvers. Since game engines mostly support skeleton based deformation, this simulated cloth/hair meshes must be retargeted to the joints that articulate those meshes. Deducing the joint rotations from a simulated mesh can be done using rivets (tracking point on mesh) or by decomposition.

In rivet scheme, first the joints are bound to the closest polygon on mesh at a rest pose and then the joint positions and rotations are evaluated from the simulated mesh at every frame and the values are keyframed. But after skinCluster evaluation using these

keyframed joint rotations and static skin weights, the mesh might not match the simulated mesh, as only the closest polygon was accounted for evaluating joint positions and rotations. This could potentially result in the re-evaluated cloth/hair meshes colliding or penetrating with other objects like body mesh (Fig. 1a).

Alternatively, we explored deducing joint rotations from deformed mesh using skinning decomposition [Le and Deng 2012]. We employed a proprietary Maya plugin (*Mesh2Bones*) developed internally at Activision that uses similar decomposition/minimization technique to find joint's positions and rotations and skin weights from a sequence of deforming geometry or simulated geometry in this case. To avoid collisions witnessed in rivet methodology, we can solve the minimization using higher resolution of joints. But we lose control over the evaluated shape of the joint chain, resulting in undesirable orientations. Fig. 1b shows evaluated joints that do not form a straight line down the cape. This limits artists from further editing the joint animations. Additionally, the minimization is computationally expensive, which impacts artists iterating over a desired setup.

2 APPROACH

To address the limitations of simulating a mesh with physical properties and then evaluating a joint chain from it, we instead explored simulating the joint chain that makes the skinned mesh deform physically correct, taking the collisions with external objects into account. So given a cloth/hair mesh, artists would set up joints based on the mesh resolution and the required simulation fidelity. We will then model the joint chains as mass-spring systems, where every joint is represented by a particle connected by a stiff spring to the next joint. Initially we simulated the chain using the Implicit mass-spring solver from Vega physics library [Sin et al. 2013]. After each simulation step, we align the joints to the simulated particles by rotating the joints iteratively from root to tip. We use the quaternions that aid in aligning the joints, to evaluate the Euler angles of the joints for key-framing.

To handle collisions of cloth/hair mesh with other meshes (body), we initially defined collision capsules along each cloth/hair joint, with radius that encompassed the associated meshes. At every simulation iteration, the collisions were resolved by penalty forces. Capsules worked effectively for specific objects like pony-tail hair, earrings, etc. To extend support to cloth and hair whose meshes are thin and flat, we replaced capsules with collision boxes (Fig. 1c). Further, we switched to Position Based method (PBD) [Müller et al. 2007] to support Continuous Collision Detection (CCD), which allowed to directly manipulate particle's position.

Our earlier setup using the nCloth solver, discussed in the introduction section, was limited in how we can independently bend and twist along the length of the meshes. To overcome that in our custom solution, we followed [Macklin et al. 2016; Umetani et al. 2014], which used ghost particles to construct material frames and employ compliant constraint formulations to decouple bending and twisting stiffness from the iteration count. At each PBD iteration, we first evaluate the stretching, bending and twisting of the joints, ignoring the collisions. Then we compute an orthogonal frame per joint using the simulated particles, which in turn is used to construct the collision boxes mentioned above. We then proceed to resolve the collisions between the boxes and the other meshes in

the scene. We also perform CCD by taking the previous time-step's collision boxes into account. At the end of the PBD iteration, we reconstruct the particles from the resolved collision boxes and iterate the process until the simulation converges.

Our custom solution yielded simulated joint chains that handled collision and produced physically appropriate motions aiding artists to quickly deliver cloth/hair animations to the game engines. However, for longer chains, the simulation took around 500 iterations to converge. To improve the performance, we incorporated quaternion based direct solver [Deul et al. 2018] which solves the stretching, bending and twisting calculations simultaneously, yielding 3-10 times speed improvement. But we witnessed jittering (Fig. 2) when the timestep is large and bending/twisting parameters are small. Instead of increasing the simulation sub-steps, we employed Long-Range Attachment (LRA) method [Kim et al. 2012] to stem the jitter. While the publication suggests using LRA after solving the stretching constraint, we used LRA before the direct solve on the constraints. This helped the simulations on long chains to converge robustly and optimally.

3 CONCLUSION

Our solution has greatly benefited animators who have applied it to a wide variety of character accessories like clothes, chains, hairs in our productions since 2015. Robust simulation, better collision handling, optimized evaluation coupled with simpler setup interface and intuitive controls have made our tool an essential part of the animator's workflow. Our artists have further requested the tool to support self-collisions, art-directable simulations among other features, which we plan to incorporate in our future work.

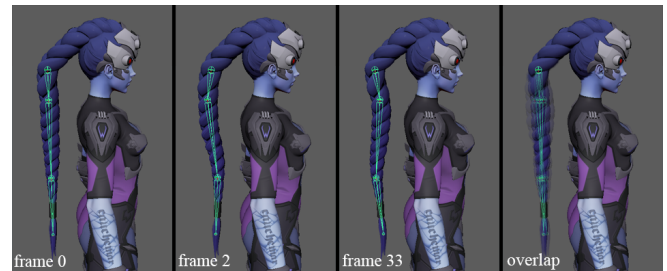


Figure 2: Without LRA, a ponytail jitters under large timestep ($\Delta t = 0.35$) and small bend/twist stiffness (10.0).

REFERENCES

- C. Deul, T. Kugelstadt, Weiler M., and J. Bender. 2018. Direct Position-Based Solver for Stiff Rods. *Computer Graphics Forum* 37, 6 (2018), 313–324.
- T.-Y. Kim, N. Chentanez, and M. Müller. 2012. Long Range Attachments - a Method to Simulate Inextensible Clothing in Computer Games. In *ACM SIGGRAPH/Eurographics Symposium on Computer Animation*. Eurographics Association, 305–310.
- B. H. Le and Z. Deng. 2012. Smooth Skinning Decomposition with Rigid Bones. *ACM Trans. Graph.* 31, 6 (2012).
- M. Macklin, M. Müller, and N. Chentanez. 2016. XPBD: Position-Based Simulation of Compliant Constrained Dynamics. In *Proceedings of the 9th International Conference on Motion in Games*. ACM, 49–54.
- M. Müller, B. Heidelberger, M. Hennix, and J. Ratcliff. 2007. Position Based Dynamics. *Journal of Visual Communication and Image Representation* 18, 2 (2007), 109–118.
- F. S. Sin, D. Schroeder, and J. Barbič. 2013. Vega: Non-Linear FEM Deformable Object Simulator. *Computer Graphics Forum* 32, 1 (2013), 36–48.
- N. Umetani, R. Schmidt, and J. Stam. 2014. Position-Based Elastic Rods. In *ACM SIGGRAPH/Eurographics Symposium on Computer Animation*. Eurographics Association, 21–30.